ARM PrimeCell

Synchronous Static Memory Controller (PL093)

Revision: r0p3-00rel0

Integration Manual

PL093 INTM 0000 C
Open Access

ARM PrimeCell Synchronous Static Memory Controller (PL093)

Revision: r0p3-00rel0

Integration Manual

Proprietary notice

Document confidentiality status

This document is Open Access (no restriction on distribution).

Product status

The information in this document is Final (information on a developed product).

Web address

http://www.arm.com

Feedback

ARM Limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on the ARM PrimeCell Synchronous Static Memory Controller

If you have any comments or suggestions about this product, please contact your supplier giving:

- The Product name
- A concise explanation of your comments.

Contents

1 ABOUT THIS DOCUMENT

1.1 Change history

Issue	Date	Change
A01	14th June, 2002	First draft
B	4th March 2004	Updated for r0p2-00rel0 release
C	24thSeptember 2004	Updated for r0p3-00rel0 release

1.2 References

This document refers to the following documents.

Ref.	Doc. No.	Title
1	ARM IHI 0011	AMBA Specification (Rev 2.0)
2	ARM DDI 0236B	ARM PrimeCell Synchronous Static Memory Controller (PL093) Technical Reference Manual
3	PL093 DDES 0000	ARM PrimeCell Synchronous Static Memory Controller (PL093) Design Manual
4	ARM DDI 0138	Example AMBA System Technical Reference Manual
5	ARM DUI 0092	Example AMBA System User Guide

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
AHB	Advanced High-performance Bus
AMBA	Advanced Micro-controller Bus Architecture
APB	Advanced Peripheral Bus
ASB	Advanced System Bus
ASIC	Application Specific Integrated Circuit
ATPG	Automatic Test Pattern Generation
IP	Intellectual Property
RTL	Register Transfer Level
SSMC	Synchronous Static Memory Controller
STA	Static Timing Analysis

2 SCOPE

This document describes how the ARM PrimeCell Synchronous Static Memory Controller (SSMC PL093) may be integrated into an AMBA-based ASIC when using a typical industry standard ASIC design flow.

3 INTRODUCTION

The ARM PrimeCell Synchronous Static Memory Controller is a reusable soft-IP block that has been developed with the prime aim of reducing time to market for ASIC development.

A carefully chosen set of design rules has been followed to allow faster integration in most ASIC design flows using standard synthesis and test tools. The implementation can easily be customised to suit specific customer requirements.

For further information on AMBA, refer to the AMBA Specification [Ref.1].

For detailed technical information and programming data on the ARM PrimeCell block, refer to the ARM PrimeCell Synchronous Static Memory Controller (PL093) Technical Reference Manual [Ref.2].

The ARM PrimeCell Synchronous Static Memory Controller (PL093) Design Manual [Ref.3] contains information about the implementation and design of the ARM PrimeCell Synchronous Static Memory Controller block that may be needed for optional customisation of this block.

The Example AMBA System Technical Reference Manual [Ref.4] and User Guide [Ref.5] describe an example micro-controller based on an ARM processor and the low-power, generic design methodology of AMBA.

4 INTEGRATION

The Synchronous Static Memory Controller block is designed to be integrated into an AMBA system via the AHB (Advanced High-Performance Bus) slave ports and the AHB Master ports. It is designed to work in systems where the SSMC, through its AHB Slave ports, can read from or write to memories. The AHB Master port is to be used in the test mode of the SSMC.

The decode logic within the AHB decoder will need to be altered to take into account the address location of the Synchronous Static Memory Controller and any other peripherals in the memory map. The base address of the Synchronous Static Memory Controller needs to be defined on a system specific basis and the decode logic altered accordingly. If AMBA TIC methodology is to be used for integration testing then any change of address location for the Synchronous Static Memory Controller block must also be reflected in the AMBA TIC test vectors, by amending the base address declaration for the Synchronous Static Memory Controller integration test program.

The TIC block (SsmcTIC) in the SSMC (PL093) is to be used for system testing through its AHB Master ports. The low gate count TIC block allows externally applied test vectors to be converted into internal AMBA bus transfers. The AMBA test philosophy allows individual modules in the system to be tested in isolation by only using transfers from the bus. It does not rely on interaction of other system elements such as a processor core.

4.1 Supported Design Flow

The ARM PrimeCell Synchronous Static Memory Controller has been designed to allow integration with other IP blocks using typical ASIC design flows. Although consideration has been given to minimisation of area (gate count) and power-consumption, ease of integration has been given higher priority to aid design reuse.

The following sections of this document address issues that may arise during ASIC system-level integration of the ARM PrimeCell Synchronous Static Memory Controller block.

An example ASIC design flow is described below:

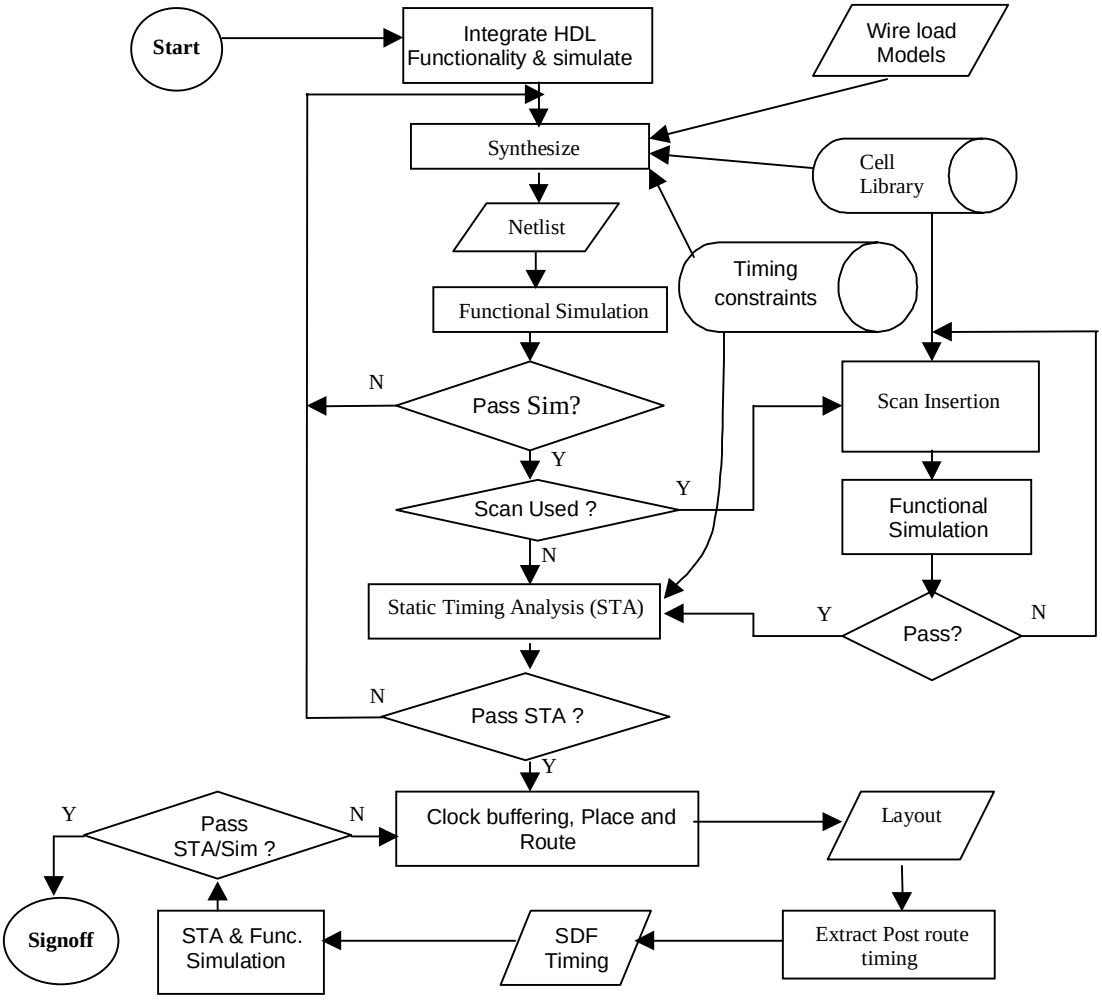

Figure 4.1-1: Example ASIC Design Flow supported

As an alternative to dynamic functional simulation it is also possible to use logic equivalence checking.

4.2 Signal Description

The following tables describe the synchronous static memory controller interface signals. ***Tables 4.2-1*** and ***Table 4.2-2*** list the AHB slave memory interface and register interface signals, while ***Tables 4.2-3*** and ***Table 4.2-4*** list the master interface signals. ***Table 4.2-5*** lists the clocks in the SSMC. ***Tables 4.2-6*** and ***Table 4.2-7*** list the SSMC Pad Interface and Pad Control signals. ***Tables 4.2-8*** and ***Table 4.2-9*** list the miscellaneous signals and the scan test related signals in the SSMC.

For more information on the AHB, please see AMBA Specification Rev2.0 [Ref. 1].

Signal Name	Type	Source / Destination	Description
HCLK Bus clock	Input	Clock Source	This clock times all bus transfers. All signal timings are related to the rising edge of HCLK.
HRESETn Reset	Input	Reset controller	The Bus reset signal is active LOW and is used to reset the system and the bus. This is the only active LOW signal.
HADDRSMC[25:0] Address bus	Input	AHB Master	The AHB address bus to the AHB Memory interface.
HWRITESMC Transfer direction	Input	AHB Master	When HIGH this signal indicates a write transfer and when LOW, a read transfer to the memory.
HSIZESMC[2:0] Transfer size	Input	AHB Master	Indicates the size of the transfer to the Memory. The SSMC AHB Slave Memory Interface supports only 8-bit, 16-bit and 32-bit accesses ‘000’ represents 8-bit ‘001’ represents 16-bit ‘010’ represents 32-bit. If HSIZEx[2:0] indicates a transfer width larger than 32-bits, the SSMC returns an ERROR response on HRESPSMC[1:0]
HTRANSSMC[1:0] Transfer type	Input	AHB Master	Indicates whether a data-transaction is to be done for the current transfer to Memory. ‘00’ represents IDLE. ‘01’ represents BUSY. ‘10’ represents NSEQ ‘11’ represents SEQ
HBURSTSMC[2:0] Burst Type	Input	AHB Master	Indicates if the transfer forms part of a burst. 4, 8 and 16 beat bursts are supported and the burst can be either incrementing or wrapping.
HREADYINSMC Transfer done	Input	AHB Slave	When HIGH the HREADYIN signal indicates that a transfer has finished on the bus. When LOW, it indicates that the AHB Slave has introduced a wait state.
HSELSSMC[7:0] Chip select-specific select	Input	AHB Decoder	Each SSMC AHB slave memory interface has its own set of chipselect -specific select signals. When HIGH, the relevant bit of HSELSSM [7:0] indicates that the current transfer is intended for a memory chip connected to a particular chip select output of the SSMC corresponding to that bit. For example, if bit 0 is asserted during an AHB transfer, it indicates that the transfer is intended for chip select 0. This signal is a combinatorial decode of the address bus.
HWDATASMC[31:0] Write data bus	Input	AHB Master	The write data bus is used to transfer data from the master to the SSMC AHB Slave Memory Interface during write operations. The SSMC AHB Slave Memory Interface supports only 8-bit, 16-bit and 32-bit accesses.
HRDATASMC[31:0] Read data bus	Output	AHB Master	The read data bus is used to read data from the SSMC AHB Slave Memory Interface. The SSMC AHB Slave Memory Interface supports only 8, 16 and 32-bit accesses.
HREADYOUTSMC Transfer done	Output	AHB Master	When HIGH the HREADYOUTSMC signal indicates that a transfer targeting the SSMC AHB Slave Memory Interface has finished on the bus. When LOW, it indicates that a wait state has been introduced by the AHB Slave
HRESPSMC[1:0] Transfer response	Output	AHB Slave	The transfer response provides additional information on the status of a transfer. Defined responses from AHB slave are, OKAY, ERROR, RETRY and SPLIT. The SSMC AHB Slave Memory Interface only drives OKAY or ERROR response.

Table 4.2-1 AHB Slave Memory Interface signals

Signal Name	Type	Source / Destination	Description
HADDRREG[11:2] Address bus	Input	AHB Master	The system address bus to the AHB register interface.
HWRITEREG Transfer direction	Input	AHB Master	When HIGH this signal indicates a write transfer and when LOW, a read transfer to the SSMC registers.
HSIZEREG[2:0] Transfer size	Input	AHB Master	Indicates the size of the transfer. **The SSMC supports only 32 bit WORD accesses to all its registers.** So all transfers to and from the SSMC controller must be 32-bit (HSIZE [2:0] = '010'). Transfer sizes other than 32-bit generate an ERROR response.
HTRANSREG Transfer type	Input	AHB Master	Indicates whether a data-transaction is to be done for the current transfer to the SSMC Registers. HTRANS [1] on the AHB bus must be connected to the single-bit-wide HTRANSREG input of the SSMC. '1' represents NSEQ or SEQ and thus a valid data transfer. '0' represents BUSY or IDLE.
HREADYINREG Transfer done	Input	AHB Slave	When HIGH the HREADYINREG signal indicates that a transfer has finished on the bus. When LOW, it indicates that a wait state has been introduced by the AHB Slave
HSELREG AHB Slave select	Input	AHB Decoder	The SSMC AHB slave register interface has its own slave select signal. This signal indicates that the current transfer is intended for the registers of the SSMC. This signal is a combinatorial decode of the address bus.
HWDATAREG[31:0] Write data bus	Input	AHB Master	The write data bus is used to transfer data from the master to the SSMC AHB Slave Register Interface during write operations. The SSMC AHB Slave Register Interface supports only 32-bit accesses.
HRDATAREG[31:0] Read data bus	Output	AHB Master	The read data bus is used to read data from the SSMC AHB Slave Register Interface. The SSMC AHB Slave Register Interface supports only 32-bit accesses
HREADYOUTREG Transfer done	Output	AHB Master	When HIGH the HREADYOUTREG signal indicates that a transfer targeting the SSMC AHB Slave Register Interface has finished on the bus. When LOW, it indicates that a wait state has been introduced by the AHB Slave
HRESPREG[1:0] Transfer response	Output	AHB Slave	The transfer response provides additional information on the status of a transfer. Defined responses from AHB slave are, OKAY, ERROR, RETRY and SPLIT. The SSMC AHB Slave Register Interface only drives OKAY or ERROR response.

Table 4.2-2 AHB Slave Register Interface signals

Signal Name	Type	Source / Destination	Description
HADDRTIC[31:0] Address bus	Output	AHB Slave	The 32-bit system address bus.
HTRANSTIC[1:0] Transfer type	Output	AHB Slave	Indicates the type of the current transfer, which can be NONSEQ, SEQ, IDLE or BUSY.
HWRITETIC Transfer direction	Output	AHB Slave	When HIGH this signal indicates a write transfer and when LOW a read transfer.
HSIZETIC[2:0] Transfer size	Output	AHB Slave	Indicates the size of the transfer. The SSMC's TIC allows 8, 16 and 32-bit transfer widths. 8-bit: HSIZE[2:0] = '000' 16-bit: HSIZE[2:0] = '001' 32-bit: HSIZE[2:0] = '010'
HPROTTIC[3:0] Protection control	Output	AHB Slave	Provides information about a bus access.
HLOCKTIC Lock information for locked transfers	Output	AHB Arbiter	This signal is used to indicate to the arbiter that the requesting master is going to perform a number of indivisible transfers and the arbiter must not grant the bus to another bus master once the locked transfer has commenced.
HBURSTTIC[2:0] Burst type	Output	AHB Slave	Indicates the type of burst transfer. **Only INCR accesses are supported by the TIC Interface.** So HBURSTTIC [2:0] will always have a value of "001" in the SSMC.
HWDATATIC[31:0] Write data bus	Output	AHB Slave	The write data bus is used to transfer data from the master to the bus slaves during write operations.
HRDATATIC[31:0] Read data bus	Input	AHB Slave	The read data bus is used to transfer data from bus slaves to the bus master during read operations.
HREADYINTIC Transfer done	Input	AHB Slave	When HIGH the HREADYINTIC signal indicates that a transfer has finished on the bus. This signal may be driven LOW to extend a transfer, by the AHB slave.

Signal Name	Type	Source / Destination	Description
HRESPTIC[1:0] Transfer response	Input	AHB Slave	The transfer response provides additional information on the status of a transfer. Four different responses are supported: OKAY, ERROR, RETRY and SPLIT. To indicate that the transfer has completed successfully, the slave uses the OKAY response. To indicate that an error has occurred, the slave uses the ERROR response. To indicate that the data is not ready, the slave uses the RETRY and SPLIT responses.

Table 4.2-3 AHB Master signals from the TIC

Signal Name	Type	Source / Destination	Description
HBUSREQTIC AHB bus request	Output	AHB arbiter	Bus request signal used by the Test Interface controller to request access to the AHB bus. When HIGH, this signal indicates that the SSMC is requesting the ownership of the bus.
HGRANTTIC AHB bus grant	Input	AHB arbiter	Grant signal generated by the arbiter. This signal indicates that the SSMC is currently the highest priority master requesting the bus. The SSMC gains bus ownership when HGRANTTIC and HREADYINTIC are high on the rising edge of HCLK.

Table 4.2-4 AHB Master bus request signals

Signal Name	Type	Source / Destination	Description
SMMEMCLK Memory clock	Input	Clock Source	SSMC Memory Clock. SSMCCLK times all external memory transfers. SSMCCLK must be synchronous to HCLK. SSMCCLK can operate at the following ratios with reference to HCLK: - 1:1 - 2:1. - 3:1. Used for Synchronous Memory devices.
nSMMEMCLK	Input	Clock Source	Inverted Memory clock

Signal Name	Type	Source / Destination	Description
SMMEMCLKDELAY	Input	Clock Source	Delayed version of SSMC Memory Clock. This clock is used to drive out the control and data signals for Synchronous Memory.
SSMCFBCLKIN3...0 Fed-back clock in	Input	Clock Source	Fed-back clocks from the Synchronous Memory devices. There are 4 such clocks – one for each byte lane. Each clock is used to register one byte of the 32-bit data returned by the Synchronous Memory device.
SMCLK[3:0] Synchronous Memory clock out	Output	Synchronous memory devices	Memory clock output. Each bit of SMCLK[3:0] is to be connected to the clock input of Synchronous Memory devices.

Table 4.2-5 SSMC clock signals

Signal Name	Type	Source / Destination	Description
nSMBLS[3:0] SSMC byte lane select	Output	Pad	Byte lanes select signals. Active LOW. The signal nSMBLS[3:0] select byte lanes [31:24], [23:16], [15:8] and [7:0] on the data bus.
nSMOEN SSMC output enable	Output	Pad	Output enable for static memories. Active LOW.
nSMWEN SSMC write enable	Output	Pad	Write enable for memories. Active LOW.
nSMCS1…7 Static memory chip select	Output	Pad	Static memory chip selects. Default active LOW.
SMCS1…7 Static memory chip select	Output	Pad	Static memory chip selects. Default active HIGH.
SMADDR[25:0] Memory address	Output	Pad	Memory Address output.
SMBAA Address advance	Output	Pad	External burst address advance signal. Used to advance address in Synchronous burst memory device.
SMADDRVALID Address valid	Output	Pad	Address valid indication for Synchronous memories.
SMDATAOUT[31:0] Write data	Output	Pad	Data output to memory. Used for the static memory controller, the synchronous memory controller and the Test Interface Controller (TIC).
SMDATAIN[31:0] Read data	Input	Pad	Read data from memory. Used for the static memory controller, the synchronous memory controller and the Test Interface Controller (TIC).
SMTESTREQA Test bus request A in test mode	Input	Pad	During test, this signal is used to indicate the type of test vector that will be applied in the following cycle. The signal gives indication that the test bus has been granted and completion of a test access. When SMTESTACK is LOW, the current test vector must be extended until SMTESTACK becomes HIGH.

Signal Name	Type	Source / Destination	Description
SMTESTREQB Test bus request B in test mode	Input	/Pad	During test, this signal is used, in combination with SMTESTREQA, to indicate the type of test vector that will be applied in the following cycle. The signal gives indication that the test bus has been granted and completion of a test access. When SMTESTACK is LOW, the current test vector must be extended until SMTESTACK becomes HIGH.
SMTESTACK Test Mode	Output	Pad	Test bus acknowledge.

Table 4.2-6 Pad Interface signals

Signal Name	Type	Source / Destination	Description
nSSMDATAEN[3:0] Data enable	Output	Pad control	Tri-State I/O pad enables for the byte lanes of the external memory data bus SSMCDATA[31:0]. Active LOW. Each bit enables the byte lanes [31:24], [23:16], [15:8] and [7:0] of the data bus independently. Used for the static memory controller and the Test Interface Controller (TIC).

Table 4.2-7 Pad Control Signals

Signal Name	Type	Source / Destination	Description
SMBIGENDIAN	Input	System Control	Static pin indicating endianness of the system. When HIGH, it indicates Big-Endian mode. When LOW, it indicates Little-Endian mode.
SMMWCS7[1:0]	Input	Pad	Memory width of CS7. This power on reset value can be over written through the register interface. “00” indicates 8-bits, “01” indicates 16-bits and “10” indicates 32-bits.
SMBLSPOL	Input	Pad	Static input pin used to program the polarity of SMBLS bit field in Bank7 control register.
SMMEMCLKRATIO[1:0]	Input	Clock Control	Static pin indicating the Clock ratios between HCLK and SMMEMCLK. This value can be overwritten by register write.
nSMBURSTWAIT	Input	Pad	Synchronous burst wait signal from external memory to delay the transfer.
SMWAIT	Input	Pad	Asynchronous wait signal from external memory controller to delay the transfer.
SMCANCELWAIT	Input	Pad	Asynchronous wait signal from external memory controller, to signal that SMWAIT is timed out.
SMBUSGNTEBI	Input	EBI	External bus granted for memory transfer.
SMTICBUSGNTEBI	Input	EBI	External bus granted for TIC transfer.

Signal Name	Type	Source / Destination	Description
SMBUSBACKOFFEBI	Input	EBI	Backoff indication from EBI for memory accesses.
SMEXTBUSMUX	Input	EBI	Static External Bus select pin. High indicates that EBI needs to be used. Low indicates that internal DBI needs to be used for arbitration.
SMBUSREQEBI	Output	EBI	Request EBI for Memory transfer.
SMTICBUSREQEBI	Output	EBI	Request EBI for TIC transfer.

Table 4.2-8 Miscellaneous signals

Signal Name	Type	Source / Destination	Description
SCANENABLE	Input	Test Controller	Scan Enable signal. Indicates that the circuit is in scan-test mode.
SCANINHCLK	Input	Test Controller	HCLK domain Scan data input for clocking in the test pattern in scan-test mode.
SCANINSMMEMCLK	Input	Test Controller	SMMEMCLK domain Scan data input for clocking in the test pattern in scan-test mode.
SCANINnSMMEMCLK	Input	Test Controller	nSMMEMCLK domain Scan data input for clocking in the test pattern in scan-test mode.
SCANINFBCLK0	Input	Test Controller	SSMCFBCLKIN0 domain Scan data input for clocking in the test pattern in scan-test mode.
SCANINFBCLK1	Input	Test Controller	SSMCFBCLKIN1 domain Scan data input for clocking in the test pattern in scan-test mode.
SCANINFBCLK2	Input	Test Controller	SSMCFBCLKIN2 domain Scan data input for clocking in the test pattern in scan-test mode.
SCANINFBCLK3	Input	Test Controller	SSMCFBCLKIN3 domain Scan data input for clocking in the test pattern in scan-test mode.
SCANINCLKDELAY	Input	Test Controller	SSMCCLKDELAY domain Scan data input for clocking in the test pattern in scan-test mode.
SCANOUTHCLK	Output	Test Controller	HCLK domain Scan data output for observing the flip-flop contents in scan test mode.
SCANOUTSMMEMCLK	Output	Test Controller	SMMEMCLK domain Scan data output for observing the flip-flop contents in scan test mode.
SCANOUTnSMMEMCLK	Output	Test Controller	nSMMEMCLK domain Scan data output for observing the flip-flop contents in scan test mode.
SCANOUTFBCLK0	Output	Test Controller	SSMCFBCLKIN0 domain Scan data output for observing the flip-flop contents in scan test mode.
SCANOUTFBCLK1	Output	Test Controller	SSMCFBCLKIN1 domain Scan data output for observing the flip-flop contents in scan test mode.

Signal Name	Type	Source / Destination	Description
SCANOUTFBCLK2	Output	Test Controller	SSMCFBCLKIN2 domain Scan data output for observing the flip-flop contents in scan test mode.
SCANOUTFBCLK3	Output	Test Controller	SSMCFBCLKIN3 domain Scan data output for observing the flip-flop contents in scan test mode.
SCANOUTCLKDELAY	Output	Test Controller	SMMEMCLKDELAY domain Scan data output for observing the flip-flop contents in scan test mode.

Table 4.2-9 Scan Test related signals

4.3 AMBA bus connection

The AHB register interface must be connected to the AHB bus with the microprocessor so that the microprocessor can program the SSMC.

Any unused AHB interfaces must have their AHB input signals tied LOW.

The TIC AHB master interface, if required, must be connected to the same bus as the microprocessor. The TIC controller must be the highest priority master on this bus. The TIC controller must not be the default master. Because the TIC master only operates at test speed, it might not function correctly if granted the bus at functional speed.

The SSMC block interfaces with the AMBA AHB interfaces using separate uni-directional read and write data buses HRDATA and HWDATA. For further information refer to the AMBA AHB Specification [Ref.1].

The AHB data buses from the SSMC should be interfaced with other data buses from other peripherals using multiplexers at the chip level.

4.4 Non-AMBA Intra-chip connection

On power on reset, the static memory bank7 memory width pins (SMMWCS7) should be driven to “00” for 8-bit, “01” for 16-bit and “10” for 32-bit databus width this value will be used until the SMBCR7 register is written into. Once the register is programmed, the memory width bits of the register will be used to determine the memory width.

On power on reset, the static memory byte lane polarity pins (SMBLS7POL) should indicate the polarity of the byte lane selects (0 for active LOW and 1 for active HIGH). These values will be used until the SMBCRx register is written into.

On power on reset, the endian mode of the SSMC should be driven on the SMBIGENDIAN pin.

On power on reset, external bus interface indicator of SSMC should be driven on the SMEXTBUSMUX pin.

The SSMC supports connection to an external bus interface (EBI) to allow external bus arbitration if there are other masters on the external bus in addition to the SSMC. SSMCEBIREQ, SSMCEBIGNT and SSMCEBIBACKOFF should be connected to the EBIREQ, EBIGNT and EBIBACKOFF signals of one port on the EBI (External Bus Interface) in such a case. In case SSMC is the only master on the external bus interface, SSMCEBIGNT should be tied to a HIGH and SSMCEBIBACKOFF should be tied to a LOW. This will ensure optimum utilisation of the external bus bandwidth.

Once the SSMC has entered test mode, it is assumed that there will be no normal mode accesses. The SSMC will enter test mode only if the sequence of events as indicated below is followed. The following sequence of events has to be used to use the SSMC’s TIC to apply test patterns:

- Apply reset (HRESETn) asynchronously.
- When the reset is removed asynchronously, the SSMC enters TIC test mode, if the SMITCR is programmed for test mode.
- The TIC block in the SSMC requests for the access to the external bus through the assertion of the SMTICBUSREQEBI signal. Once SMTICBUSGNTEBI is sampled asserted, the TIC block takes control of the external bus and relinquishes the bus only after it has completed all the transfers on the AHB master interface.

The TIC block asserts its HBUSREQTIC signal to the arbiter, requesting access to the AHB bus. Because the TIC is the highest priority, HGRANTTIC is asserted and the granted TIC takes ownership of the AHB bus.

For more related information please refer to ARM PrimeCell Synchronous Static Memory Controller Technical Reference Manual [Ref. 4].

4.5 Primary input/output connection

The primary inputs and outputs of SSMC are the external memory bus signals. These signals should be connected to the appropriate pins of the external memory device. The following points have to be kept in mind while connecting the memory devices to the SSMC.

4.5.1 Write enable connection of static memory devices

- For memory banks constructed from 8-bit or 16-bit memory devices, the nSMBLS[3:0] signals should be connected to write enable (nWE) inputs of each 8-bit memory. The nSMWEN signal from the SSMC should not be used.
- For memory banks constructed from 16-bit or 32-bit memory devices, the nSMBLS[3:0] signals should be connected to the byte select inputs of each memory device. The nSMWEN signal from the SSMC should be connected to the write enable inputs of each memory device.

4.5.2 Address connection of static memory device

When external memory width is 8-bits or 16-bits or 32-bits, SMADDR [25:0] should be connected to the A[25:0] pins of the static memory device.

4.6 Clocks

The ARM PrimeCell SSMC HDL contains 7 clock sources at the top level – HCLK, SSMCCLK, SSMCCLKDELAY, SSMCFBCLKIN0, SSMCFBCLKIN1, SSMCFBCLKIN2 and SSMCFBCLKIN3. SMMEMCLK and HCLK should be synchronous and rising edge aligned in 1:1, 2:1 and 3:1 clocking modes.

HCLK is the AHB clock used for the AHB side signals and for controlling the accesses to static memory registers.

SMMEMCLK is used for the memory controller logic in the SSMC. This clock is also used to derive the clock inputs of the synchronous memory device, SMCLK[3:0]. Each bit of SMCLK[3:0] is used to drive the synchronous memory devices connected to one of memory chip select.

Ideally SMMEMCLKDELAY should be inverted version of SMMEMCLK for all purpose.

SMCFBCLKIN[0..3] are the clocks fed-back from the memory devices to clock in the read data from the memories. Each of these clocks is used to clock in data for one byte-lane of the memory read data. SSMCFBCLKIN0 is used to clock in SMDATAIN[7:0], SMCFBCLKIN1 is used to clock in SMDATAIN[15:8] and so on.

Please refer to the ARM PrimeCell Synchronous Static Memory Controller (PL093) Technical Reference Manual [Ref 2] for more details on the clocks in the SSMC.

4.7 Resets

The SSMC block has one reset pin – HRESETn. HRESETn is the system bus reset.

HRESETn can be asserted asynchronously by the system reset controller, but they should be deasserted synchronous to the rising edge of HCLK.

4.8 Scan Signals

SCANENABLE is the scan-mode enable pin of the SSMC. A separate scan input and output is provided for each of the clock domains. Hence the seven clocks lead to seven scan-chains in the SSMC. The various scan signals are described in ***Table 4.2-10***.

4.9 Synchronisation

HCLK and SMMEMCLK, the main clock inputs of the SSMC are assumed, synchronous and rising edge aligned to each other. Therefore the design does not require any synchronization logic.

4.10 Endianness

The Synchronous Static Memory Controller supports up to 32-bit accesses on its AHB slave data buses. For a 32-bit system, the Synchronous Static Memory Controller AHB slave interface is bi-endian to the extent that it can support 32-bit accesses from a little-endian master and a big-endian master. Memory accesses with HSIZE[2:0] less than the memory databus width are mapped to the correct byte lanes of data in the memory devices depending on the endian mode of the SSMC. For AHB memory accesses to memories that have 8-bit or 16-bit databus widths, data packing and unpacking is done according to the endian mode of the SSMC. Please refer to the ARM PrimeCell Synchronous Static Memory Controller (PL093) Technical Reference Manual [Ref 2] for more details on the address and data translation for AHB memory accesses depending on the endian mode.

In systems with wider data buses, data routing will have to be done external to the SSMC. The endianness of the AHB Slave ports of the SSMC is pin controlled. The SMBIGENDIAN pin can be used to control the endianness of the AHB slave ports.

5 SYNTHESIS

The Synchronous Static Memory Controller block has been synthesised using the Synopsys Design Compiler synthesis tool with the scripts located in the synopsys directory.

A README text file in the synopsys directory provides details of the synthesis directory structure used.

The master compile script - Ssmc.cmd - synthesises the Synchronous Static Memory Controller to the chosen target cell library. If scan-insertion is to be performed the master compile script invokes the Ssmc_scan.cmd script.

The final area and gate count that will be achieved is dependent on the target cell library used, and synthesis timing constraints and command options.

5.1 Constraints

The SSMC is synthesised to 133MHz.

The constraints on the different inputs and outputs of the SSMC are described in the following sub-sections.

5.1.1 AMBA bus signals

The AMBA signals are synthesised at the following constraints:

- 30 % input setup on all inputs to the AHB Slave ports and the AHB Master port
- 40 % output valid delay on all outputs of the AHB Slave ports and the AHB Master port

5.1.2 Non-AMBA Intra-chip signals

The Non-AMBA Intra-chip signals are synthesised at the following constraints:

- 30% input setup on all non-AMBA Intra-chip inputs.
- 40% output valid delay on all non-AMBA Intra-chip outputs.

5.1.3 Primary input/ output signals

All primary inputs and outputs are synthesised at the following constraints

- 30% input setup on all primary inputs.
- 40% output valid delay on all primary outputs.

A false path is set on SMCLK to avoid the false violations reported on this path. Also, a false path is set from the HREADYINTIC pin of the Ssmc module since, SsmcTIC module is synthesised at 10 MHz and the logic within this module should not be optimised during the synthesis of the top-level.

5.1.4 Clocks

Many different design methodologies are available for clock tree insertion and synthesis. The default Synchronous Static Memory Controller synthesis scripts cause the Synopsys tools to output a netlist with no buffering on the clock nets. It is assumed that a separate tool, such as place & route tools, which understand placement, further on in the backend process, will perform clock buffering. It is possible to modify the scripts if this is not the preferred

approach or if this is not compatible with the design flow being used. It is recommended however that clock tree insertion should always be performed during the place and route stage and not during synthesis.

5.1.5 Resets

One of several available methods should be used to ensure correct buffering of the reset nets. The reset signals could be assigned infinite drive strength allowing a chip-level integration process to ensure balanced reset trees are used at the top-level of the chip or propagating the tree down to all end points. Another alternative is to apply timing constraints and ensure that block synthesis instantiates cells with adequate drive strength in each block

5.1.6 Scan signals

False paths are set from all scan related inputs so that the tool should not try to optimise the scan paths.

5.2 Synthesis scripts

The synthesis scripts consist of the following three main categories:

5.2.1 Setup Scripts

Scripts that are common to several designs are located in the $GLOBAL/synopsys_shared/scr_common directory. [Please refer to **Appendix A** for the list of PrimeCell world environment variables]. The following files need several parameters or variables to be changed in order to target a different cell library.

The setup script, setup.scr defines the following:

- Synopsys variable settings
- UNIX directory path for Synopsys read and write caches

The global.scr script may need modifying to define:

- target area goals
- maximum transition costs

The library_avanti_cb18_v1.0.scr script is included by the master compile script to specify the target library and the cells in the target library that should not be used. It is recommended that an equivalent file be created for the target library as per library vendor or customer requirements. It is also a practice to sometimes prevent the synthesis tool from inferring lower drive cells for timing reasons. The user must ensure that the library_avanti_cb18_v1.0.scr file is modified and renamed appropriately for the chosen technology library and included by the master compile script.

The $LIB_SYNOP directory must contain the files, or links to the target cell library files, in Synopsys format.

5.2.2 Command Script

The master compile command script Ssmc.cmd in the synopsys/scripts directory analyses the HDL source before mapping and optimising to a specific target technology cell library. The Ssmc.cmd script includes setup and constraint scripts to set timing and design rule constraints on the design, prior to optimisation. The compiled design is written out both in native Synopsys .db format and in VHDL/Verilog netlist format. Pre-route (post-synthesis) timing information for back-annotation is written out to SDF files and various reports for timing and violations are also written out. Additionally the run-time log is redirected into a separate log/ directory.

The scan-insertion script Ssmc_scan.cmd in the synopsys/scripts directory performs scan insertion for the design.

Prior to synthesis, several variables need to be set. The scripts use environment variables, which are assigned to synthesis variables in the scripts. The environment variables provide flexibility without requiring editing of the scripts. The following environment variables are used:

- $PERIPH is assigned to ‘topname’ to define the name of the top-level module in the design.
- $HDL_SOURCE is assigned to ‘hdl’ to define the input HDL source. HDL_SOURCE should be set to either vhdl or verilog in order to read in the VHDL or Verilog source RTL files.
- $HDL_NETL is assigned to ‘hdl_out’ to define the output format for netlist and SDF files. It can be set to either vhdl or verilog to generate either VHDL or Verilog netlists.
- $TEST_METH is assigned to ‘test_req’ in setup.scr. This can be set to ‘noscan’ or ‘scaninsert’.
- $GLOBAL is assigned to ‘synth_base_area’ and is used to locate the global area, where global primecell scripts and a cache area are located.

If the environment variables are not required, then the scripts can be modified to directly assign values.

5.2.3 Constraint Scripts

a) Operating conditions and signal load/drive values should be set for the chosen technology library in a file equivalent to the library_avanti_cb18_v1.0.scr file.

b) Timing constraints for AHB bus signals are set via the amba_params.scr, ahb_master.scr and the ahb_slave.scr files. The SSMC is compiled at 133 MHz. These timings should be adjusted according to the target frequency required. The above-mentioned files have been written using parameters and only requires amendment of the clock frequency with all other bus parameters (except clock skew) scaling accordingly.

   Clock skew needs to be set explicitly as minus (worst case) and plus (best case) uncertainty. The following default values have been used as preliminary values prior to layout:

   - The minus skew is set at 2.5% of the clock period. This is subtracted from the clock period used for the synthesis
   - The plus skew is set at 40% of minus skew. This is used for the plus uncertainty value.

   The plus skew derives from best case condition and it is less than the minus skew, which derives from worst case condition.

   The timing parameters assume the following bus timing budgets:

   - 30 % input setup on all input ports including those on the AHB Slave ports, the AHB Master port and the non-AMBA input lines.
   - 40 % output valid delay on all outputs including those on the AHB Slave ports and the AHB Master port
   - 40% output valid delay on all the non-AMBA output lines.

c) Timing constraints for non-AMBA SSMC-specific signals are set via the Ssmc.scr file. Sample input and output delay constraints are specified for these signals. The minimum delays are set so that the synthesis tool does not insert buffers to fix hold violations that do not exist in reality.

d) Timing exceptions are specified via the Ssmc_exceptions.scr file. False paths are specified to SMCLK[3:0] since it is the gated clock output of the SSMC and timing constraints are not set for these output clocks.

6 INTEGRATION TEST

The Synchronous Static Memory Controller integration test vector suite allows the integrator to verify that the Synchronous Static Memory Controller has been connected into the target system correctly.

6.1 Integration Test strategy

Integration tests are required to test three classes of signals:

6.1.1 AMBA signals

The AHB signals connected to the AHB Slave ports of the Synchronous Static Memory Controller signals are tested with AHB transactions to the SSMC. The vectors required for testing the AHB register port are present in the IntegrationTest.c file in the integration/bustest directory.

6.1.2 Non-AMBA intra-chip signals

Running the Integration test vectors present in the IntegrationTest.c file in the integration/bustest directory tests the non-AMBA intra-chip signals of the SSMC.

6.1.3 Primary inputs and outputs

The primary inputs and outputs (external memory signals) can be tested external to the chip by performing memory transactions. Running the test vectors present in the IntegrationTest.c file in the verification/bustest directory can do this.

6.2 Integration Test Code

The integration/bustest directory contains the integration test code files, IntegrationTest.c and IntRegisterTest.c and the verification/bustest directory contains the file IntegrationTest.c.

The integration/bustest directory also contains the tests to verify the SSMC's TIC block. The test vectors to test the SSMC's TIC module are in the ResponseTests.c and TransferTests.c files. The file SSMC.c in the integration/bustest directory is used to selectively run the integration tests or TIC tests or all of these tests. As delivered, the integration test vectors are intended to run correctly with the Synchronous Static Memory Controller in stand-alone mode within the integration EASY world i.e. even before the Synchronous Static Memory Controller has been integrated into the user system.

After integration, the following points need to be noted:

- The integration vectors targeting the Synchronous Static Memory Controller AHB slave port (AMBA signals) connectivity does **NOT** have to be modified by the integrator.
- The integration test vectors targeting the intra-chip signals of the SSMC are required to be modified to drive a '1' or a '0' on the intra-chip inputs and to read the intra-chip outputs. The test vectors that need to be modified have been indicated in the IntegrationTest.c file in the /integration/bustest directory.

7 PRODUCTION TEST

The Synchronous Static Memory Controller is designed to be production tested using scan-insertion and ATPG.

7.1 Scan Insertion and ATPG

It is assumed that the user will already be familiar with the design flow for scan insertion and ATPG, so only a brief discussion of integration issues arising are given below.

The Synchronous Static Memory Controller design fully supports this scan test and ATPG methodology. Depending on the clocking arrangement used, the number of chip level scan chains required may vary. As delivered, the Synchronous Static Memory Controller is synthesised with 7 scan-chains.

If scan insertion is to be performed post-synthesis, it should be done once the netlist has been proven to be logically equivalent either by dynamic simulation using the functional test vectors or through use of equivalence checking. Scan insertion is then performed and the logical equivalence again proven. For scan insertion using a two-pass approach, the RTL is compiled into a netlist without scan cells in the first pass and in the second pass, the normal storage elements are replaced with scan cells and the scan chains are formed. Alternatively, a one-pass approach may be adopted for synthesis where the RTL is compiled to create netlist with a scan test D flips-flops and provisionally connected scan chains. After scan insertion logical equivalence should again be proven.

The system designer does extraction of test vectors once the whole chip design has been integrated. The quoted fault coverage (excepting variation resulting from the use of a sparse cell library) will be met by extracting test vectors around the complete chip while running the automatically generated vectors (ATPG).

8 FUNCTIONAL AND TIMING VERIFICATION

It is essential to verify the functionality and the timing performance of the netlist against the RTL at key stages in the design flow, post-synthesis, post-scan insertion, during layout and at completion before sign-off of the ASIC

8.1 Functional verification

Functional verification is used to test the functionality of the design. The netlist should also be checked for functional equivalence to the RTL at all the key stages of the design flow. The functional verification can be performed by one of the two methods specified below:

8.1.1 Formal Verification

Formal verification is used to compare RTL with RTL and RTL with netlist. The functionality of the netlist or RTL can be compared to that of the RTL by use of a logic equivalence-checking tool. (For example, Synopsys Formality or Chrysalis). Functionality of the netlist and RTL can be compared within a shorter period of time by this method when compared to gate-level simulations.

8.1.2 Dynamic simulation

Verification tests are run on the RTL and netlist to confirm functionality of the design. Gate-level simulations can be used to compare the netlist functionality to that of the RTL. The same test vectors that were used to verify the RTL are used to verify the netlist functionality. This is however a time consuming process.

8.2 Timing verification

It is essential to verify timing performance of the netlist against the RTL at all the key stages of the design flow. Use of timing verification tools is a useful check for human error in the manual definition of false paths. The following two methods can be used to perform timing verification:

8.2.1 Static Timing analysis

Static timing verification is a worthwhile and efficient method of verifying timing performance of a design block, but can tend to be pessimistic unless false paths are defined to exclude such paths from analysis.

8.2.2 Dynamic simulation

Gate-level simulation can be used for limited timing verification but it is slow and cannot exercise all timing paths in the design. Results tend to be optimistic since worst case timing paths may not be exercised.

9 APPENDIX A

9.1 PrimeCell Environment Variables

Simulation and Synthesis in the Synchronous Static Memory Controller world are controlled by a set of environment variables.

GLOBAL

The path to the shared files is defined by the **'GLOBAL'** variable.

e.g. :

- **setenv GLOBAL /h/arm/global**

PERIPH

The peripheral name is defined by the **'PERIPH'** variable.

e.g.:

- **setenv PERIPH Ssmc**

SIMULATOR

The simulator type is defined by the **'SIMULATOR'** variable.

It can take the following two values

- modelsim - for ModelSim simulations
- LDV - for LDV simulations

e.g.:

- **setenv SIMULATOR modelsim**
- **setenv SIMULATOR LDV**

TEST_METH

The scan methodology adopted during synthesis is defined by the **'TEST_METH'** variable.

It can take the following two values

- noscan - creates a netlist without scan structures
- scaninsert - creates a fully routed scan netlist.

e.g. :

- **setenv TEST_METH noscan**
- **setenv TEST_METH scaninsert**

HDL_SOURCE

The source code type is defined by the **'HDL_SOURCE'** variable.

It can take the following two values

- vhdl [VHDL RTL]
- verilog [Verilog RTL]

e.g. :

- **setenv HDL_SOURCE vhdl**
- **setenv HDL_SOURCE verilog**

HDL_NETL

The netlist type is defined by the **'HDL_NETL**' variable, and it is always verilog

e.g. :

- **setenv HDL_NETL verilog**

TEST_ENV

The test environment is defined by the **'TEST_ENV**' variable.

It can take the following two values

- BUSTALK [For functional vector simulations]
- TICTALK [For integration vector simulations]

e.g. :

- **setenv TEST_ENV BUSTALK**
- **setenv TEST_ENV TICTALK**

LIB_VHDL

The location of the vhdl cell library views is defined by the **'LIB_VHDL**' variable.

e.g. :

- **setenv LIB_VHDL /AVANT/cb18/v1.0/vital/cb18os120/zero**

DIRS_VERILOG

The location of the verilog cell library views is defined by the **'DIRS_VERILOG'** variable.

e.g. :

- **setenv DIRS_VERILOG /AVANT/cb18/v1.0/verilog/cb18os120/zero**

FILES_VERILOG

The location of the verilog UDP (User Defined Primitive) file is defined by the **'FILES_VERILOG'** variable.

e.g. :

- **setenv FILES_VERILOG /AVANT/cb18/v1.0/verilog/cb18os120/zero/mtb_verilog.v**

LIB_SYNOP

The location of the synopsys target library files is defined by the **'LIB_SYNOP'** variable.

e.g.:

- **setenv LIB_SYNOP /AVANT/cb18/v1.0/synopsys/cb18os120/1999.05/models**

AMBA

This environment variable defines the AMBA bus to which the peripheral is connected - AHB, ASB or APB. Since the SSMC PL093 is an AHB peripheral, set this environment variable to 'AHB'. This environment variable is used by makefiles in the SSMC database to select settings for the Integration world.

e.g. :

- **setenv AMBA AHB**